

RentaMovil Software Technical Proposal Documentation

Nicole Dayana Ramirez Vargas – Thiago Fabian Rojas Guevara – David Santiago Erez

Rodriguez

Servicio Nacional de Aprendizaje (SENA)

Training program: Software Analysis and Development

Course or cohort: 3145556

Submission date: October 9, 2026 – Preparation date: October 9, 2026

Abstract

This document proposes and justifies RentaMovil's technical solution before its detailed design: the alternatives evaluated for each relevant architectural decision, the chosen technology stack, the high-level architecture, project feasibility, the development methodology, and a proposed schedule by epic. It does not replace the architecture decision records (ADR) in the documentation repository, which remain the source of truth: it summarizes them, relates them to each other, and presents them as a single, coherent proposal for evaluators and the development team.

Keywords: RentaMovil, technical proposal, microservices architecture, hexagonal architecture, architectural decisions, feasibility, Scrum methodology.

1. Introduction

1.1 Purpose, audience, and scope

This document is addressed to instructors, evaluators, and the development team. Its purpose is to consolidate, in a single document, the technical decisions already recorded as ADRs (Architecture Decision Records) in the *rtm-docs* repository, together with the proposed technology stack, feasibility, and planning for the project. It covers the microservices backend, the client and administrative web application, and the client mobile application.

1.2 Objectives

General objective: propose and justify RentaMovil's technical solution, documenting the alternatives evaluated, the chosen architecture, and its feasibility for the team and the time available within the training program.

Specific objectives:

- Record the alternatives considered for each relevant architectural decision and the reason for the final choice.
- Justify the proposed technology stack for the backend, web frontend, mobile application, and database.
- Assess the project's technical, operational, and economic feasibility.
- Describe the development methodology agreed upon by the team.
- Propose a high-level schedule and the identified risks with their mitigation.

1.3 Relationship to the other project documents

This document sits between the requirements specification (document 1) and the dynamic-views analysis document (document 2), which establish *what* the system must do, and the

software design documentation (document 4), which details *how* it is built (graphical interfaces, architectural model, patterns, and data model). The technical proposal answers the intermediate question: *what solution was chosen and why*, before moving into detailed design (RentaMovil, 2026, 01-context/overview.md; 03-product/vision.md).

2. Solution Alternatives Evaluated

Every relevant architectural decision in the project was recorded as an ADR (Architecture Decision Record) under *05-architecture/decisions/records/*. This section summarizes, for each one, the chosen alternative, the alternatives discarded, and the reason for the choice. All decisions described here have *Proposed* status in the repository: they are the project's target architecture, not necessarily the current state of every line of code.

2.1 Backend decomposition into microservices

Alternative	Status	Reason
One service per Bounded Context (7 services)	Chosen	Aligns code ownership with the domain language and the events already defined in the domain model
One service per frontend feature	Discarded	Couples backend boundaries to the user interface and duplicates domain ownership
One large, undefined monolith	Discarded	Does not establish independent domain ownership

The trade-off of the chosen option is that migrations on the shared SQL Server database require explicit coordination between services, and independent deployment of each service remains a future capability, not a current one (RentaMovil, 2026, ADR-002-microservices-decomposition.md).

2.2 Mobile client architecture

Alternative	Status	Reason
Feature-Based + Component-Driven + Expo Router	Chosen	Clear ownership by business capability and reuse of UI components
Folders organized only by layer	Discarded	Spreads each feature's behavior across the whole project
Business logic inside route files	Discarded	Couples navigation to business behavior

As a result, new mobile client features live under *src/features/*, shared UI under *src/shared/*, and *src/app/* is reserved for navigation only (RentaMovil, 2026, ADR-003-mobile-architecture.md).

2.3 Client state management

Alternative	Status	Reason
Local state + feature Context + Zustand for cross-feature state	Chosen	Explicit state ownership and gradual adoption, without forcing a single global mechanism
Zustand for all client state	Discarded	Encourages unnecessary global coupling
Context for all client state	Discarded	Can cause broad re-renders and unclear ownership

Server-owned data stays in the backend services and is not treated as a source of truth on the client; this decision does not authorize direct client access to persistence (RentaMovil, 2026, ADR-004-state-management.md).

2.4 Web authentication and authorization

Alternative	Status	Reason
Backend authorization + frontend route guards	Chosen	Secure and usable; the final access decision always belongs to the backend

Alternative	Status	Reason
Authorization resolved only on the frontend	Discarded	Does not protect the data exposed by the API
Supabase Auth as the identity issuer	Discarded	Conflicts with the project's security policy

The web frontend integrates with the identity service through the backend API, stores refresh tokens in *httpOnly* cookies, and uses route guards for navigation; access tokens are issued as JWTs signed with RS256 (RentaMovil, 2026, ADR-005-web-authentication-and-authorization.md).

2.5 Web frontend architecture

Alternative	Status	Reason
Standardized feature structure (Feature + Component + Route-Based)	Chosen	Predictable and scalable as the number of screens grows
Preserve every existing folder convention	Discarded for new code	Keeps the risk of inconsistent imports on case-sensitive systems
Project structure organized only by layer	Discarded	Weakens each feature's ownership

The web frontend previously mixed different folder conventions (*Pages/pages*, *Services/services*); new code adopts lowercase names and a plural *services/*, migrating the rest incrementally (RentaMovil, 2026, ADR-006-web-frontend-architecture.md).

2.6 Frontend-backend boundary

Alternative	Status	Reason
Clients consume only the backend's own REST APIs	Chosen	Centralizes business rules and authorization in the backend

Alternative	Status	Reason
Direct frontend access to Supabase or another data provider	Discarded as target architecture	Bypasses domain ownership and the backend's security policy
Direct frontend access to SQL Server	Discarded	Insecure and impossible to govern

Any temporary use of Supabase in current clients is treated as technical debt or a temporary integration point, and must not be expanded without updating the corresponding ADR (RentaMovil, 2026, ADR-007-frontend-backend-boundary.md).

2.7 MVP route scope

Alternative	Status	Reason
Hide or remove rental-contract routes from the frontend	Chosen	Aligns active navigation with the MVP's real scope
Keep the existing contract routes active	Discarded	Misrepresents the MVP scope and creates flows the backend does not support
Add contracts to the MVP informally	Discarded	Breaks scope governance and traceability to the requirements

The route catalog distinguishes public, client, administrative, and legacy routes; adding rental contracts to the scope would require an approved scope change and updates to product, requirements, domain, API, and architecture documentation (RentaMovil, 2026, ADR-008-mvp-route-boundary.md).

2.8 Database schema and migrations

Alternative	Status	Reason
One schema per service + one migration repository per service	Chosen	Each table's ownership is explicit in its name; repositories never collide with each other

Alternative	Status	Reason
All tables in <i>dbo</i> , one migration repository per service	Discarded	Ownership remains only by convention; any service's DB user can write to any table
A single migration repository shared by every service	Discarded	Couples every team's release to a single repository
An independent database per service	Discarded	Contradicts the shared database and loses the foreign keys between contexts

The database remains a single shared SQL Server instance (*rentamovil*), with one schema per business service (*iam, fleet, telemetry, booking, payment*) and one *rtm-<service>-db* repository per service. The execution order between repositories is fixed (*iam* → *fleet-maintenance* → *telemetry-gps* → *booking-reservation* → *payment-billing*) so that cross-service foreign keys always point to tables that already exist (RentaMovil, 2026, ADR-009-schema-per-service-and-migration-repos.md).

3. Proposed Technology Stack

Layer	Technology	Justification
Backend	Java + Spring Boot, Hexagonal Architecture	Separates business rules from Spring, SQL Server, and transport; the training program's standard
Database	Microsoft SQL Server (shared instance, one schema per service)	The project's single declared persistence technology; migrated with Liquibase per service
Web (client and administration)	React + Vite	Enables a feature-based, component-reusable architecture with fast build times
Mobile (client)	Expo + React Native	A single source for Android/iOS, with Expo Router for navigation

Layer	Technology	Justification
Authentication	JWT signed with RS256, issued by the identity service	Lets every microservice verify the token independently, without querying the issuer on every request
Communication	Synchronous REST between client and backend; asynchronous domain events between services, delivered <i>at-least-once</i>	Requires the payment, rental, telemetry, and notification flows to be idempotent

(RentaMovil, 2026, 05-architecture/overview.md)

4. Proposed High-Level Architecture

The architecture is client-server: the client web and mobile applications, and the administrative web interface, consume the backend's APIs through published (OpenAPI) contracts. The backend is organized into seven Bounded Contexts, grouped into deployable services:

Bounded Context	Target Service
Identity & Access	identity-access-service
Fleet & Maintenance	fleet-maintenance-service
Booking & Reservation	booking-reservation-service
Rental Execution	rental-execution-service (runs inside booking-reservation)
Payment & Billing	payment-billing-service
Telemetry & GPS	telemetry-gps-service
Notification	notification-service (runs inside booking-reservation)

audit is cross-cutting infrastructure written directly by all seven contexts, not its own Bounded Context with a dedicated screen or microservice.

Architectural principles: API contracts before implementing public endpoints (*API first*); domain isolation from Spring, SQL Server, and transport; ownership bounded by context; controlled dependencies only via REST or published domain events, never direct internal access; security built in by design; incremental system evolution.

Architectural patterns: Hexagonal Architecture (adopted in the backend), Feature-Based and Component-Driven (adopted in the mobile client), API Gateway (the system's target boundary), Saga/Outbox and Circuit Breaker (to be evaluated for cross-context flows and calls between independently deployed services, respectively). An eventual migration from the shared database to a database per service would require a new ADR (RentaMovil, 2026, 05-architecture/overview.md).

5. Feasibility

5.1 Technical feasibility

The team has prior experience with Java/Spring Boot, React, and relational databases, reinforced during the training program. The chosen stack (section 3) uses tools in widespread industry use with abundant documentation, which reduces adoption risk. The main technical complexity identified is coordinating migrations across the *rtm-<service>-db* repositories, mitigated by the fixed execution order described in section 2.8.

5.2 Operational feasibility

The project is developed by a team of 3 members under the methodology described in section 6, using free tools or tools with a sufficient free tier (Jira, GitHub, Docker). No production infrastructure dependencies beyond a local or containerized development and testing environment were identified (RentaMovil, 2026, 00-governance/agile-conventions.md).

5.3 Economic feasibility

As an academic project, there is no budget allocated for production infrastructure or commercial licenses. The proposed technology stack (section 3) uses exclusively open-source tools or tools with a free tier sufficient for the MVP's scope (Java, Spring Boot, SQL Server Developer Edition, React, Expo, Docker). No formal cost-benefit analysis of a production deployment was performed, as it falls outside the training program's academic scope.

6. Development Methodology

The team agreed to work under **Scrum**, with **one-week** sprints (Monday through Friday) for a **3-person** team. Ceremonies were compressed to fit the short sprint length:

Ceremony	Timing	Maximum Duration	Participants
Sprint Planning	Monday	1 hour	Full team
Daily Stand-up	Daily	15 minutes	Full team
Backlog Refinement	Wednesday	30 minutes	Full team
Sprint Review	Friday	30 minutes	Team + Product Owner
Sprint Retrospective	Friday (after the Review)	30 minutes	Full team

Every retrospective must produce at least one improvement action with an owner and a due date. Story estimation uses the **Fibonacci** scale (1, 2, 3, 5, 8, 13) via **Planning Poker**; a story estimated at 8 or 13 points must be split before entering a sprint. The backlog is managed in **Jira**, with a board of columns Backlog → Ready → In Progress → In Review → Done (RentaMovil, 2026, 00-governance/agile-conventions.md).

7. Required Resources

7.1 Human resources

The team is made up of 3 members, with the roles already defined in the project's requirements specification:

Member	Role	Responsibility
Nicole Dayana Ramírez Vargas	Lead	Lead, plan, and oversee the project delivery
David Santiago Erez Rodríguez	Developer	Build the web application
Thiago Fabian Rojas Guevara	Analyst	Information analysis, design, and programming

(RentaMovil, 2026, Document 1 — Software Requirements Specification, section 1.7)

7.2 Software resources

Category	Tools
Backend	Java (JDK), Spring Boot, Maven/Gradle
Database	Microsoft SQL Server Developer Edition, Liquibase
Web frontend	Node.js, React, Vite
Mobile application	Expo, React Native
Version control	Git, GitHub
Backlog management	Jira
Containers	Docker
Development environments	Team IDEs (IntelliJ IDEA, Visual Studio Code)

Every tool listed is open source or has a free tier sufficient for the MVP's scope, consistent with the economic feasibility described in section 5.3.

7.3 Hardware resources

The project does not require production servers: development, testing, and integration run on the team members' own computers and local Docker containers. Mobile application testing runs on emulators (Android/iOS) or the team's own physical devices, with no additional

infrastructure. The exception is the Telemetry & GPS module: to validate real integration with a GPS device, the team purchased a physical module and its components, itemized in the budget in section 7.4.

7.4 One-Time Materials and Expenses

The following are the expenses the team has incurred to date, covered out of pocket by its members, mainly to physically validate the Telemetry & GPS module:

Item	Description	Approximate cost (COP)
GPS module and materials	GPS device and electronic components for the tracking prototype	\$300,000
Additional electronic supplies	Cables, connectors, breadboard, and enclosure	\$50,000
Transportation	Travel to purchase materials	\$50,000
Contingency and miscellaneous	Additional unplanned expenses during purchase and integration	\$100,000
One-time expenses subtotal		\$500,000

7.5 Recurring Costs and Total Estimated Project Cost

Taking the project's total duration as **18 months** (the team's full time in the training program, not only the sprint-execution weeks in section 8), the following recurring monthly costs are also estimated: each team member's salary and the internet and electricity services used during development.

Item	Monthly cost (COP)	Months	Total cost (COP)
Team salary (3 members × \$800,000)	\$2,400,000	18	\$43,200,000
Services (internet and electricity, whole team)	\$150,000	18	\$2,700,000

Item	Monthly cost (COP)	Months	Total cost (COP)
Recurring costs subtotal			\$45,900,000
Summary	Cost (COP)		
One-time expenses (section 7.4)	\$500,000		
Recurring costs (18 months)	\$45,900,000		
Total estimated project cost	\$46,400,000		

These figures are an academic estimate, meant to illustrate the real cost that the team's time would represent during the training program; they do not correspond to a production-deployment budget, which falls outside the program's academic scope, as already noted in the economic feasibility (section 5.3).

8. Current Status and Proposed Schedule

The project backlog, documented in *04-requirements/user-stories.md*, is organized into 7 epics (one per Bounded Context) with 31 user stories in total, estimated between 2 and 5 points each.

This proposal suggests executing the epics in the same dependency order that already governs database migrations between services (section 2.8), so that no service depends on tables or contracts that do not yet exist:

Order	Epic (Bounded Context)	Dependency	Estimated Duration
1	Identity & Access	None — authentication is required by everything else	2 sprints
2	Fleet & Maintenance	Identity & Access (roles and permissions)	2 sprints
3	Telemetry & GPS	Fleet & Maintenance (vehicles to track)	1 sprint

Order	Epic (Bounded Context)	Dependency	Estimated Duration
4	Booking & Reservation	Identity & Access, Fleet & Maintenance	2 sprints
5	Rental Execution	Booking & Reservation (confirmed reservation)	1 sprint
6	Payment & Billing	Booking & Reservation	2 sprints
7	Notification	Cross-cutting across the previous epics' events	1 sprint
Estimated total			11 sprints (≈ 11 weeks)

With one-week sprints and a 3-person team, the full project is estimated at roughly 11 weeks of execution, from the first Sprint Planning through the closing of the last epic. Per-epic duration is a planning estimate, proportional to its relative complexity; the team does not yet have recorded real velocity data (RentaMovil, 2026, 00-governance/agile-conventions.md), so no concrete calendar dates are set — the start and end of each sprint remain subject to each week's Sprint Planning, per the methodology described in section 6.

9. Risks and Mitigation

Risk	Source	Proposed Mitigation
Uncoordinated schema migrations across services sharing the database	ADR-002, ADR-009	Fixed execution order between migration repositories and mandatory review before deploying schema changes
Residual Supabase references in the frontend	ADR-007	Treat them as explicit technical debt; do not expand them without updating the corresponding ADR

Risk	Source	Proposed Mitigation
Rental-contract routes exposed outside the MVP scope	ADR-008	Hide or remove active navigation to those routes until an approved scope change exists
Inconsistent folder conventions in the web frontend (letter casing)	ADR-006	Require the standardized convention for new code; migrate existing code incrementally
Backlog with unassigned sprint schedule	04-requirements/user-stories.md	Prioritize assigning sprints following the dependency order proposed in section 7
Estimation disagreement among team members	00-governance/agile-conventions.md	Planning Poker with a mandatory-discussion rule whenever estimates differ by 2 or more levels on the Fibonacci scale

10. Conclusions

The technical solution proposed for RentaMovil is backed by nine architectural decisions already recorded and evaluated against concrete alternatives, with a technology stack in widespread industry use and feasible for a three-member team within the training program's timeframe. The backlog's 31 user stories are already refined; sprint execution follows the dependency order between epics proposed in this document.

References

- RentaMovil. (2026). 01-context/overview.md and 03-product/vision.md [Project vision and context]. RentaMovil documentation repository.
- RentaMovil. (2026). ADR-002-microservices-decomposition.md [Architecture decision record]. RentaMovil documentation repository.

RentaMovil. (2026). ADR-003-mobile-architecture.md [Architecture decision record].
RentaMovil documentation repository.

RentaMovil. (2026). ADR-004-state-management.md [Architecture decision record].
RentaMovil documentation repository.

RentaMovil. (2026). ADR-005-web-authentication-and-authorization.md [Architecture
decision record]. RentaMovil documentation repository.

RentaMovil. (2026). ADR-006-web-frontend-architecture.md [Architecture decision record].
RentaMovil documentation repository.

RentaMovil. (2026). ADR-007-frontend-backend-boundary.md [Architecture decision
record]. RentaMovil documentation repository.

RentaMovil. (2026). ADR-008-mvp-route-boundary.md [Architecture decision record].
RentaMovil documentation repository.

RentaMovil. (2026). ADR-009-schema-per-service-and-migration-repos.md [Architecture
decision record]. RentaMovil documentation repository.

RentaMovil. (2026). 05-architecture/overview.md [Architecture overview]. RentaMovil
documentation repository.

RentaMovil. (2026). 00-governance/agile-conventions.md [Team agile conventions].
RentaMovil documentation repository.

RentaMovil. (2026). 04-requirements/user-stories.md [User stories and backlog status].
RentaMovil documentation repository.